MACHINE LEARNING SPECIALIZATION · CHEAT NOTES

吴恩达机器学习 精简复习笔记 · 打印版

只保留三类东西：**知识重点**、**考试要点**、**代码演示**。三个 Course 末尾各附一页「公式笔记」，考前刷这一份就够。**明亮配色 + A4 分页优化**：Ctrl / Cmd + P 直接打印，或选「存储为 PDF」。

C1 监督学习：回归与分类 | C2 高级学习算法 | C3 无监督·推荐·强化学习 | 共 10 章 · 30+ 公式卡

COURSE 1 / 3

监督学习：回归与分类 Supervised ML · Regression & Classification

线性回归 → 梯度下降 → 多变量 → 逻辑回归 → 过拟合与正则化。这是整套课程的数学地基。

COURSE 1 · WEEK 1–2

导论与单变量线性回归

◆ 知识重点

- **机器学习定义**（Mitchell）：任务 T、经验 E、性能 P——用 E 改善 P。
- **监督学习**：有标签；**回归**＝预测连续值，**分类**＝预测离散类别。无监督：聚类、降维。
- 训练集记号：样本 $x^{(i)}$、目标 $y^{(i)}$、样本数 m。
- 线性回归模型 $f_{w,b}(x) = wx + b$；训练＝**arg min J(w,b)**。
- 平方误差代价 J 是 (w,b) 的**凸函数**（碗形）→ 梯度下降必达全局最优。
- 「批量(batch)」梯度下降：每步用**全部** m 个样本算梯度。

* 考试要点

- **同步更新**：w、b 必须用旧值同时更新；先用新 w 去算 b 的梯度＝错。
- α 太大 → 震荡/发散；太小 → 慢。接近极小值时步幅**自动变小**（坡度变缓）。
- 收敛判据是**梯度＝0**，不是 J＝0。
- 回归 vs 分类看**输出是否连续**（房价→回归，垃圾邮件→分类）。

$$J(w, b) = \frac{1}{2m} \sum_{i=1..m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$ 代价函数

$$\text{repeat } \{\ w := w - \alpha \cdot \frac{\partial J}{\partial w};\ \ b := b - \alpha \cdot \frac{\partial J}{\partial b}\ \} \Rightarrow\ \ w := w - \alpha \cdot \frac{1}{m} \Sigma \left(f(x^{(i)}) - y^{(i)}\right) \cdot x^{(i)}$$ 梯度下降

linear_regression.py *代价 + 批量梯度下降（同步更新）* NumPy

```python
def compute_cost(x, y, w, b):
    m = len(x)
    return np.sum((w*x + b - y)**2) / (2*m)

def gradient_descent(x, y, w, b, alpha, iters):
    for _ in range(iters):
        err = w*x + b - y                # 残差基于旧参数 → 天然同步更新
        w -= alpha * np.sum(err*x) / len(x)
        b -= alpha * np.sum(err)   / len(x)
    return w, b
```

COURSE 1 · WEEK 2–3

多变量线性回归

◆ 知识重点

- 多特征模型：$f_{w,b}(x) = w \cdot x + b = w_1x_1 + \ldots + w_nx_n + b$。
- **向量化**：`np.dot(w,x)` 快于 Python for 循环（并行 + 缓存友好）。
- **特征缩放**：各特征量纲悬殊时（如 $x_1 \in [0,2000]$, $x_2 \in [1,5]$）等高线变椭圆 → 收敛慢；缩放后近圆形 → 快。
- **z-score 标准化**：$(x-\mu)/\sigma$，目标范围约 [−1, 1] 即可。
- **学习率调试**：J 应逐迭代下降；上升/震荡 → $\alpha \div 10$；太慢 → $\alpha \times 3$。
- **特征工程**：长×宽→面积；**多项式回归**：用 x, x^2, x^3 作特征拟合曲线。

✱ 考试要点

- z-score 公式必须**手算会**：给 μ、σ 算标准化值（高频选择题）。
- 判断「要不要缩放」：看特征**取值范围差几个数量级**。
- J 曲线上升 ⇒ α **过大**，第一反应是减小 α。
- 多项式回归中 x^n 范围爆炸 ⇒ **更需要**特征缩放。

multivariate.py *向量化 + z-score + 多元梯度下降* NumPy

```python
def zscore(X):                          # 逐列标准化
    return (X - X.mean(axis=0)) / X.std(axis=0)

def gd_multi(X, y, w, b, alpha, iters):
    m = X.shape[0]
    for _ in range(iters):
        err = X @ w + b - y             # f = w·x + b, 向量化
        w -= alpha * (X.T @ err) / m
        b -= alpha * err.mean()
    return w, b
```

COURSE 1 · WEEK 3

逻辑回归、过拟合与正则化

◆ 知识重点

- **Sigmoid**：$g(z) = 1/(1+e^{-z})$，输出解释为 $P(y=1|x)$。
- **决策边界**：$w \cdot x + b = 0$；多项式特征 → 非线性边界。
- **交叉熵损失**（分段）：y=1 时 $-\log f$，y=0 时 $-\log(1-f)$。
- **过拟合**（高方差）：训练好、泛化差；**欠拟合**（高偏差）：训练都差。
- 对策：①更多数据 ②减少特征 ③**正则化**（保留全部特征、缩小参数）。
- 正则化项 $\lambda/(2m)\cdot\Sigma w_j^2$：**只罚 w 不罚 b**。

* 考试要点

- 逻辑回归**不用平方误差**：非凸，梯度下降会陷入局部最优。
- 损失直觉：$f\to 0$ 而 $y=1 \Rightarrow L\to\infty$（confidently wrong 惩罚最重）。
- 逻辑回归的梯度更新公式与线性回归**形状完全相同**，只是 f 换成 sigmoid。
- λ 过大 → 所有 w→0 → **欠拟合**（接近常数预测）；λ=0 → 无正则化。

$$J(w,b) = -\frac{1}{m}\Sigma\left[\, y^{(i)}\cdot\log f(x^{(i)}) + (1-y^{(i)})\cdot\log(1-f(x^{(i)}))\,\right] + \frac{\lambda}{2m}\Sigma_{j=1..n}\, w_j^2$$

正则化逻辑回归

logistic.py *sigmoid + 正则化交叉熵 + 梯度下降* NumPy

```python
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def cost_logreg(X, y, w, b, lam=0):
    f = sigmoid(X @ w + b)
    ce = -np.mean(y*np.log(f) + (1-y)*np.log(1-f))
    return ce + lam/(2*len(y)) * np.sum(w**2)   # 不含 b

def gd_logreg(X, y, w, b, alpha, iters, lam=0):
    m = len(y)
    for _ in range(iters):
        err = sigmoid(X @ w + b) - y
        w -= alpha * (X.T @ err + lam*w) / m   # 只多出 lam*w 一项
        b -= alpha * err.mean()
    return w, b
```

Course 1 · 公式笔记

回归与分类的 10 个核心公式。⭐ = 必背。

⭐ 线性回归模型 F1.1

$$f_{w,b}(x) = w \cdot x + b$$

n 维特征时 w·x 为向量内积。

⭐ 平方误差代价 F1.2

$$J(w,b) = \frac{1}{2m} \Sigma \, (f(x^{(i)}) - y^{(i)})^2$$

系数 ½ 纯为求导消 2；**凸函数**。

⭐ 线性回归梯度更新 F1.3

$$w_j := w_j - \alpha \cdot \frac{1}{m} \Sigma \, (f - y^{(i)}) \cdot x_j^{(i)} \;\; ; \;\; b := b - \alpha \cdot \frac{1}{m} \Sigma \, (f - y^{(i)})$$

对所有参数**同步**更新。

⭐ Sigmoid F1.4

$$g(z) = 1 \, / \, (1 + e^{-z})$$

z→∞ ⇒ g→1；z=0 ⇒ g=0.5。

⭐ 逻辑回归分段损失 F1.5

$$L(f,y) = -\log(f) \;\text{ if } y=1; \;\; -\log(1-f) \;\text{ if } y=0$$

等价简并式：**−y·log f −(1−y)·log(1−f)**。

⭐ 逻辑回归代价 F1.6

$$J = -\frac{1}{m} \Sigma [\, y \cdot \log f + (1-y) \cdot \log(1-f) \,]$$

凸；梯度形式与线性回归**全同**。

z-score 特征缩放 F1.7

$$x_j' = (x_j - \mu_j) \, / \, \sigma_j$$

μ=均值，σ=标准差；让各特征量纲可比。

⭐ 正则化线性回归 F1.8

$$J = \frac{1}{2m} \Sigma (f-y)^2 + \frac{\lambda}{2m} \Sigma \, w_j^2$$

w 的更新多出 **+(λ/m)w** 项（权重衰减）。

正则化逻辑回归 F1.9

$$J = \text{交叉熵} + {}^{\lambda}\!/_{2m}\,\Sigma\, w_j^2$$

同一个 λ 框架套两种模型。

λ 的权衡 F1.10

$\lambda \to 0$：无正则(过拟合)；λ 过大：$w \to 0$ (欠拟合)

正则化**从不惩罚 b**——考试原题。

COURSE 2 / 3

高级学习算法 Advanced Learning Algorithms

神经网络、训练技巧、偏差/方差诊断、决策树与随机森林——求职面试与考试双高频区。

COURSE 2 · WEEK 1

神经网络与前向传播

◆ 知识重点

- 神经元：$a = g(w \cdot x + b)$；一层＝一组神经元同时计算。
- **前向传播**逐层计算：$a^{[l]} = g(W^{[l]} a^{[l-1]} + b^{[l]})$。
- **矩阵维度**：$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$，$b^{[l]} \in \mathbb{R}^{n^{[l]}}$。
- 三层典型结构：输入层 → 隐藏层（提取特征）→ 输出层。
- TensorFlow：`Sequential([Dense(25,'sigmoid'), …])`；`Dense` ＝一层。
- 每个神经元都在**看上一层的全部输出**——层的本质是矩阵乘法＋激活。

* 考试要点

* 给网络结构求 $W^{[2]}$ 的 shape：**(本层神经元数, 上层神经元数)**。
* 没有激活函数（g=恒等）⇒ 多层网络塌缩为**线性回归**——激活引入非线性。
* 「隐藏层」＝既非输入也非输出的层；**层数不算输入层**（常见陷阱题）。
* 前向传播手算题：一层层代 $g(Wx+b)$，别跳步。

$$a^{[2]} = g(W^{[2]} \cdot a^{[1]} + b^{[2]}),\quad a^{[1]} = g(W^{[1]} \cdot x + b^{[1]})$$

前向传播

forward_prop.py *TF 搭网络 + NumPy 手写一层* — TF · NumPy

```python
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Dense(25, activation='sigmoid'),
    tf.keras.layers.Dense(15, activation='sigmoid'),
    tf.keras.layers.Dense(1,  activation='sigmoid'),   # 二分类输出
])

# NumPy 手写 dense 层：W 形如 (n_out, n_in)
def dense(a_in, W, b, g):
    return g(a_in @ W.T + b)

a1 = dense(X, W1, b1, sigmoid)     # 逐层前向传播
a2 = dense(a1, W2, b2, sigmoid)
```

COURSE 2 · WEEK 2

激活函数、Softmax 与训练细节

◆ 知识重点

- 训练三步曲：`compile(loss)` → `fit(X,y,epochs)` → `predict`。
- 激活函数：ReLU $max(0,z)$；sigmoid；linear $g(z)=z$；softmax。
- Softmax：$a_j = e^{z_j}/\Sigma_k e^{z_k}$，输出 K 类概率分布（和为 1）。
- 多分类损失：稀疏交叉熵 $-log\ a_y$。
- Adam：自适应缩放各参数学习率（大梯度小步、小梯度大步），α 好调。
- 反向传播＝**计算图上的链式法则**，算出 ∂J/∂w（TensorFlow 自动完成）。

✱ 考试要点

- **输出层激活选择表（必考）**：二分类 sigmoid；回归（±∞）linear；非负回归 ReLU；多分类 softmax。
- 隐藏层**默认 ReLU**：计算快 + 不易梯度饱和。
- softmax 所有输出相加恒为 1；单个 z 增大 → 自身概率↑、其余↓。
- `from_logits=True`：输出层写 **linear**，数值更稳，预测时再 softmax。
- Adam vs 普通 GD：不同参数**各自不同**的有效学习率。

$$a_1,\ldots,a_K = \text{softmax}(z)_j = e^{z_j} / \Sigma_{k=1..K}\, e^{z_k} \quad L(f, y) = -\log a_y$$

softmax + 交叉熵

softmax_mnist.py *多分类 + Adam + from_logits* — TensorFlow

```
from tensorflow.keras import Sequential, Dense
from tensorflow.keras.losses import SparseCategoricalCrossentropy

model = Sequential([
    Dense(25, activation='relu'),
    Dense(15, activation='relu'),
    Dense(10, activation='linear'),          # 输出 logits
])
model.compile(loss=SparseCategoricalCrossentropy(from_logits=True),
              optimizer=tf.keras.optimizers.Adam(1e-3))
model.fit(X, y, epochs=20)

p = tf.nn.softmax(model(X)).numpy()      # 预测时手动 softmax
```

COURSE 2 · WEEK 3

模型评估、偏差/方差与改进策略

◆ 知识重点

- 数据三分：**train 60% / cv 20% / test 20%**；用 **cv 集**选模型，**test 只碰一次**。
- **高偏差**（欠拟合）：train 误差 ≈ cv 误差，且都**高于基线**。
- **高方差**（过拟合）：train 误差低，cv 误差**显著更高**。
- 基线＝人类/竞品水平；差距才有意义。
- 改进路径：更大网络→治偏差；更多数据→治方差；正则化↑→治方差。
- **迁移学习**：预训练模型＋小数据 → 只重训输出层/最后几层。
- 倾斜数据看 **P / R / F1**，不看 accuracy。

✱ 考试要点

- 诊断题套路：先看 **train 误差**（高→高偏差），再看 **cv−train 差距**（大→高方差）。
- 高偏差时「加数据」**没用**——经典陷阱选项。
- $F1 = 2PR/(P+R)$ 手算（P=TP/(TP+FP)，R=TP/(TP+FN)）。
- 正样本占比 0.1% 时，全预测负类 accuracy=99.9% 但毫无价值。
- 迁移学习适用：新任务输出层不同、数据少、与预训练域相近。

症状	train err	cv err	处方
高偏差	高	≈ train（高）	更大网络 · 加特征 · 降 λ
高方差	低	远高于 train	更多数据 · 减特征 · 升 λ
两者皆高	高	更高	先治偏差，再迭代

diagnose.py *60/20/20 划分 + 用 cv 选多项式阶数* sklearn

```python
from sklearn.model_selection import train_test_split

X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.4)
X_cv, X_te, y_cv, y_te = train_test_split(X_tmp, y_tmp, test_size=0.5)

for d in [1, 2, 3, 4]:                          # 用 cv 集选模型，不碰 test
    poly = PolynomialFeatures(d).fit(X_tr)
    m = LinearRegression().fit(poly.transform(X_tr), y_tr)
    print(d, f"cv={m.score(poly.transform(X_cv), y_cv):.3f}")

P, R = TP/(TP+FP), TP/(TP+FN)           # 倾斜数据
F1 = 2*P*R / (P + R)
```

COURSE 2 · WEEK 4

决策树、随机森林与 XGBoost

◆ 知识重点

- 决策树：每节点选**信息增益最大**的特征切分，递归到纯/达停止条件。
- **熵**：$H(p) = -p\cdot log_2 p -(1-p)\cdot log_2(1-p)$；p=0.5 时最大=1。
- **信息增益** = 父节点熵 − 加权子节点熵（权重=样本占比）。
- 类别特征 → **one-hot**；连续特征 → 阈值二分；回归树 → 按**方差减少**分裂。
- **随机森林**：bagging（有放回抽 m 个样本）＋每节点**随机特征子集**（√n），多树投票。
- **XGBoost**：boosting——新树专注拟合**前一棵树的残差**。

✱ 考试要点

- **熵计算必考**：H(0.5)=1，H(0)=H(1)=0，H(0.8)≈0.72。
- 信息增益手算：先算父熵，再算左右子熵加权平均，**相减**。
- 随机森林 > 单树：集成**降低方差**、不易过拟合。
- bagging vs boosting：并行独立 vs **串行纠错**。
- XGBoost 为什么强：正则化 + 早停 + 并行优化，表格数据首选。

$$IG = H(p_{node}) - \left(w^{left}\cdot H(p^{left}) + w^{right}\cdot H(p^{right}) \right),\quad w = {}^{n_{子}}\!/_{n_{父}}$$

信息增益

trees.py *熵 / 信息增益手算 + 随机森林* NumPy · sklearn

```
def H(p):                                  # 二分类熵
    return 0 if p in (0, 1) else -p*np.log2(p) - (1-p)*np.log2(1-p)

def info_gain(p_root, p_l, p_r, w_l):
    return H(p_root) - w_l*H(p_l) - (1-w_l)*H(p_r)

from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, max_features='sqrt')
rf.fit(X_train, y_train)                   # bagging + 特征子集

import xgboost as xgb                      # 表格数据首选
bst = xgb.XGBClassifier().fit(X_train, y_train)
```

● Course 2 · 公式笔记

前向传播、softmax、熵与信息增益、P/R/F1——考试计算题全在这一页。

前向传播 F2.1

$$a^{[l]} = g(W^{[l]}\, a^{[l-1]} + b^{[l]})$$

l 层输入 = 上层输出 $a^{[l-1]}$。

参数维度 F2.2

$$W^{[l]} : (n^{[l]} \times n^{[l-1]}) \;;\; b^{[l]} : (n^{[l]})$$

shape 判断题直接套。

ReLU F2.3

$$g(z) = \max(0, z)$$

隐藏层默认；快、无右端饱和。

Softmax F2.4

$$a_j = e^{z_j} / \Sigma_k\, e^{z_k}$$

$\Sigma a_j = 1$；配合稀疏交叉熵。

多分类交叉熵 F2.5

$$L = -\log a_y \quad (J = {}^{1}\!/_{m}\, \Sigma\, L)$$

只看真实类别的概率。

熵 F2.6

$$H(p) = -p\cdot\log_2 p - (1-p)\cdot\log_2(1-p)$$

H(0.5)=1 最大；H(0)=H(1)=0。

信息增益 F2.7

$$IG = H(父) - [\, w_L H(左) + w_R H(右)\,]$$

决策树每步最大化的量。

精确率 / 召回率 F2.8

$$P = TP/(TP+FP) \;;\; R = TP/(TP+FN)$$

P 治「误报」，R 治「漏报」。

F1 F2.9

$$F1 = 2PR / (P + R)$$

P、R 的调和平均，倾斜数据主指标。

输出层激活速查 F2.10

二分类→sigmoid ; 回归→linear ; 非负→ReLU ; K类→softmax

选择题原题库。

COURSE 3 / 3

无监督学习、推荐系统与强化学习 Unsupervised, Recommenders & RL

K-means、高斯异常检测、协同过滤、PCA，最后以 MDP 与 DQN 收官。

COURSE 3 · WEEK 1

K-means 聚类与异常检测

◆ 知识重点

- **K-means 两步循环**：① 分配——每个样本归最近质心 $c^{(i)} = argmin\|x-\mu\|$；② 移动——质心＝簇内均值。
- 代价：$J = (1/m)\Sigma\|x^{(i)}-\mu_{c(i)}\|^2$，两步都在降 J。
- 初始化：随机挑 K 个训练样本；**跑 50–1000 次取 J 最小**。
- 选 K：肘部法则（不总有效）/ 下游任务评估。
- **异常检测**：拟合 $p(x)$，$p(x) < \varepsilon$ 判异常。
- 高斯模型：$p(x) = \Pi_j\, p(x_j; \mu_j, \sigma_j^2)$，各维独立。

✱ 考试要点

- K-means 必收敛（J 单调不增），但**不保证全局最优**——靠多次初始化。
- 高斯公式手算 + **σ 大 → 曲矮胖**，σ 小 → 曲瘦高。
- **选异常检测 vs 监督学习**（高频）：负样本**极少**且未来异常**模式未知** → 异常检测；两类样本都多 → 监督学习。
- 非高斯特征 → **log 变换**再看直方图。
- ε 靠 **P/R 权衡**（真异常 vs 误报）调。

$$p(x_j) = \frac{1}{\sqrt{(2\pi)\cdot\sigma_j}} \cdot \exp\left(-\frac{(x_j-\mu_j)^2}{2\sigma_j^2}\right) \quad p(x) = \Pi_{j=1..n}\, p(x_j)$$

高斯密度

unsupervised.py *KMeans + 高斯异常检测* — sklearn · NumPy

```python
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, n_init=10).fit(X)      # n_init=多次初始化
labels, centers = km.labels_, km.cluster_centers_

def gaussian(x, mu, sig):                        # 一维高斯
    return np.exp(-(x-mu)**2/(2*sig**2)) / (np.sqrt(2*np.pi)*sig)

mu, sig = X.mean(axis=0), X.std(axis=0)          # 各维独立估计
p = np.prod(gaussian(X, mu, sig), axis=1)
anomaly = p < 0.002                              # 阈值 ε 由 P/R 调
```

COURSE 3 · WEEK 2

协同过滤、内容过滤与 PCA

◆ 知识重点

- **协同过滤**：用户参数 w 与物品特征 x **同时学习**（评分矩阵已知，特征未知）。
- 代价：只对**已评分** (r=1) 的 (user,item) 对求和，＋双方正则项。
- **均值归一化**：新用户（无评分）预测＝全站均值，避免全 0。
- **基于内容过滤**：物品特征 x 已知（如电影题材），学用户参数；深度版：两个网络提 v_u, v_m，点积打分。
- **PCA**：找 k 个正交方向，投影方差最大 / 重构误差最小。
- 流程：中心化 → 协方差 $\Sigma=(1/m)X^TX$ → 特征向量 → $z = U_{reduce}{}^T x$。

✱ 考试要点

- **协同过滤 vs 内容过滤**：特征已知→内容；特征要学→协同（一句话必考）。
- 均值归一化的动机：**冷启动用户给均值**而非零。
- PCA 选 k：**保留 ≥99% 方差**（(ΣS[:k])/ΣS ≥ 0.99）。
- PCA 用途：加速、可视化、降噪；**不推荐**用于防过拟合（正则化更好）。
- PCA 必须先**中心化**（缩放可选但常见）。

$$J(w,x) = \Sigma_{r(i,j)=1} \left(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \lambda \left(\Sigma\Sigma w^2 + \Sigma\Sigma x^2 \right)$$

协同过滤代价

reco_pca.py *协同过滤梯度下降 + PCA 三步* NumPy

```
# 协同过滤：W(用户) 与 X(物品) 同时梯度下降，R∈{0,1} 标记已评分
for _ in range(iters):
    e = R * (X @ W.T + b - Y)            # 只算已评分位置
    W -= alpha * (e.T @ X + lam * W)
    X -= alpha * (e @ W   + lam * X)

# PCA：中心化 → SVD → 取前 k 维
Xc = X - X.mean(axis=0)
U, S, Vt = np.linalg.svd(Xc)
Z = Xc @ Vt[:k].T                        # 压缩：z = UᵀX
X_rec = Z @ Vt[:k]                       # 重构
keep = S[:k].sum() / S.sum()             # 保留方差比，目标 ≥ 0.99
```

强化学习：MDP、Q-Learning 与 DQN

◆ 知识重点

- 四元组框架：状态 S、动作 A、奖励 R、折扣 γ；策略 $\pi(a|s)$。
- **回报**：$R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$；γ=0.9 → 10 步后权重 < 1/3。
- **状态值** $V(s) = max_a\, Q(s,a)$；**最优策略** $\pi(s)=argmax\, Q$。
- **Bellman 方程**：$Q(s,a) = R(s,a) + \gamma \cdot max_{a'}\, Q(s',a')$。
- 随机环境 → 取**期望**；连续状态（如月球着陆器）→ **DQN**。
- DQN 训练目标：$y = R + \gamma \cdot max_{a'}\, Q_{target}(s',a')$，拟合 $Q(s,a)$。
- 技巧：**ε–greedy** 探索（ε 随时间衰减）、小批经验回放、目标网络定期同步。

✱ 考试要点

- **折扣回报手算**（必考）：γ=0.5、奖励 [1, 2, 3] → 1+0.5·2+0.25·3=2.75。
- γ 越小越**短视**；γ=0.99 → 看约百步，γ=0.9 → 看约十步。
- Bellman 应用题：给 Q 表算 $Q(s,a)$ 或 $V(s)$。
- $Q(s,a)$ 含义＝在 s 做 a、之后**一直最优**的期望回报。
- ε–greedy：概率 ε 随机（探索），其余贪心 argmax（利用）。

$$Q(s,a) = R(s,a) + \gamma \cdot \max_{a'} Q(s',a') \quad \Rightarrow \quad \pi^*(s) = \operatorname{argmax}_a Q(s,a)$$

Bellman 方程

q_learning.py *表格型 Q-learning + DQN 训练核心* — NumPy · TF

```python
# 表格型：离散小状态空间
Q = np.zeros((n_states, n_actions))
for ep in range(3000):
    s = env.reset()
    for t in range(100):
        a = np.random.choice(A) if np.random.rand() < eps else Q[s].argmax()
        s2, r, done = env.step(a)
        Q[s,a] += alpha * (r + gamma*Q[s2].max()*(not done) - Q[s,a])
        s = s2
        if done: break
    eps *= 0.995                          # 探索率衰减

# DQN 核心：网络拟合 Q(s,·)，目标来自目标网络
q_net   = Sequential([Dense(64, 'relu'), Dense(64, 'relu'), Dense(n_act)])
tgt_net = tf.keras.models.clone_model(q_net)       # 定期同步
y = r + gamma * tgt_net(s2).max(axis=1) * (1 - done)   # Bellman 目标
```

Course 3 · 公式笔记

聚类、高斯、协同过滤、PCA 与强化学习——最后一页收官公式。

☆ K-means 代价 F3.1

$$J = \frac{1}{m} \Sigma \|x^{(i)} - \mu_{c(i)}\|^2$$

分配、移动两步均单调降 J。

质心更新 F3.2

$$\mu_k = \text{mean}\{\, x^{(i)} : c^{(i)} = k \,\}$$

簇内样本均值。

☆ 高斯密度 F3.3

$$p(x) = \frac{1}{\sqrt{(2\pi)}\sigma}\, e^{-(x-\mu)^2/2\sigma^2}$$

μ 定位置，σ 定胖瘦。

☆ 异常判据 F3.4

$$p(x) = \Pi_j\, p(x_j;\, \mu_j,\, \sigma_j^2) < \varepsilon$$

ε 由 P/R 权衡决定。

☆ 协同过滤代价 F3.5

$$J = \Sigma_{r=1}(w \cdot x + b - y)^2 + \lambda(\Sigma w^2 + \Sigma x^2)$$

w、x **同时**梯度下降。

PCA 压缩 / 重构 F3.6

$$z = U_{reduce}{}^{T}\, x \;;\; \hat{x} = U_{reduce}\, z$$

先中心化；方向＝协方差特征向量。

☆ PCA 选 k F3.7

$$\Sigma_k S \,/\, \Sigma S \geq 0.99$$

保留 99% 方差所需的最小 k。

☆ 折扣回报 F3.8

$$G = R_0 + \gamma R_1 + \gamma^2 R_2 + \ldots$$

γ=0.9≈看十步；手算必考。

☆ Bellman 方程 F3.9

$$Q(s,a) = R(s,a) + \gamma \cdot \max_{a'} Q(s',a')$$

V(s)=max Q；π*=argmax Q。

☆ Q-learning 更新 F3.10

$$Q(s,a) \leftarrow Q + \alpha[\, R + \gamma \cdot \max_{a'} Q(s',a') - Q \,]$$

DQN：Q 换成神经网络，配 ε-greedy。